Proyecto 3: La bodega del Tiburón Dorado

Tomas Santana

Noviembre 2026

1 La historia

El *Tiburón Dorado* es el barco más rápido del Caribe, pero también el más desordenado. Durante la última tormenta, el contramaestre perdió el libro donde anotaba todo lo que había en la bodega, y desde entonces nadie sabe cuántas galletas quedan, ni si todavía hay pólvora para los cañones.

La capitana Rita Sinmiedo está cansada de perder doblones en cada puerto. Te ha reclutado como la persona de la tripulación que sabe programar, con una misión: escribir un programa que lleve la cuenta de la bodega. Con él, la tripulación podrá vender mercancía en los puertos, reabastecerse, registrar el botín que capturen y, al final del viaje, revisar la bitácora para saber cuánto ganaron y qué se está acabando.

Si el programa falla, la capitana ya advirtió que el responsable limpiará la cubierta durante un mes.

2 Descripción

Se debe desarrollar un programa en C++ que administre la mercancía de la bodega del *Tiburón Dorado*: la muestra, registra ventas, reabastecimientos y botines, y genera una bitácora con los doblones ganados, el valor de la bodega y la mercancía que escasea. Todos los montos se expresan en **doblones**.

El proyecto se entrega **completo en una sola fecha**, pero está pensado para desarrollarse en dos partes:

Parte	**Qué se construye**	**Se puede hacer después de**
Parte 1	La bodega con arreglos paralelos: menú, ventas, reabastecimiento y bitácora.	Lab 6: arreglos, recorridos, búsqueda y arreglos en funciones.
Parte 2	La bodega con un arreglo de `struct`: nombres de la mercancía, búsqueda por nombre y registro de botines.	Lab 7: `struct` y arreglos de `struct`.

> **Recomendación:** terminar la Parte 1 antes de la clase del Lab 7. En la Parte 2 no se empieza de cero: se reorganiza el programa de la Parte 1 para usar un `struct`.

3 Evaluación

La nota del proyecto tiene dos componentes: la implementación del programa y la explicación del mismo en clase.

$$\text{Nota final} = \text{Implementación} \times \text{Explicación}$$

Componente	Escala	Qué se evalúa
Implementación	0 a 20	Que el programa compile, cumpla todas las reglas y produzca los resultados correctos.
Explicación	0 a 1	Que el estudiante explique su código, prediga su comportamiento y lo modifique en clase.

> **Importante:** la explicación multiplica la nota. Un programa perfecto (20) que el estudiante no puede explicar (0) tiene una nota final de **0**. Un programa de 16 explicado con seguridad (1) tiene una nota final de **16**.

3.1 Implementación (0 a 20)

Parte	Criterio	Puntos
—	El programa compila sin errores con `g++` y se entrega a tiempo.	2
1	Menú principal y listado de la bodega (secciones 4.2 y 4.3).	2
1	Ventas en puerto: búsqueda por código, validaciones y actualización de existencias (sección 4.4).	4
1	Reabastecimiento (sección 4.5).	2
1	Bitácora: doblones ganados, valor de la bodega, mercancía más vendida y mercancía que escasea (sección 4.6).	3
2	Arreglo de `struct` en lugar de arreglos paralelos y funciones obligatorias (secciones 5.1 y 5.4).	3
2	Búsqueda por nombre (sección 5.2).	1
2	Registro de botines con sus validaciones (sección 5.3).	2
—	Legibilidad: nombres descriptivos, indentación y comentarios.	1
	Total	**20**

El programa se probará con los casos de la sección 6 y con casos adicionales que no aparecen en este documento. Un programa que no compila tiene 0 en implementación.

3.2 Explicación (0 a 1)

Durante la presentación, con el programa abierto en VS Code, el profesor hará preguntas sobre el código. Las preguntas pueden consistir en:

- Explicar qué hace una parte específica del código y por qué se escribió de esa manera.
- Indicar qué mostrará el programa con ciertos datos antes de ejecutarlo.
- Realizar en el momento un cambio pequeño en las reglas y demostrar que funciona (por ejemplo, agregar un campo al `struct` o cambiar el límite de mercancía que escasea).

Valor	Criterio
1	Explica cualquier parte con seguridad, predice la salida correctamente y realiza el cambio solicitado.
0.75	Explica casi todo; duda en algún detalle o necesita una pista pequeña para realizar el cambio.
0.5	Explica la idea general, pero no justifica sus decisiones ni logra realizar el cambio.

Valor	Criterio
0.25	Reconoce partes del código, pero no explica cómo funciona.
0	No puede explicar su propio código.

> Cualquier ayuda recibida (compañeros, internet, herramientas de IA) debe ser declarada, según lo establecido en el sílabo. En todos los casos, el estudiante debe comprender cada línea de su programa: la presentación es individual.

4 Parte 1: La bodega con arreglos

4.1 Mercancía inicial

Al zarpar, la bodega tiene esta mercancía, ya cargada en el código:

Código	Mercancía	Precio (doblones)	Existencias
101	Galletas	1.50	20
102	Cuerdas	1.20	15
103	Velas	1.00	4
104	Pólvora	3.50	8
105	Mapas	1.30	0
106	Brújulas	4.00	10

En la Parte 1 se usan **arreglos paralelos**, como en el ejercicio de carnets del Lab 6: `codigos`, `precios`, `existencias` y `vendidas`. La posición `i` de cada arreglo corresponde a la misma mercancía. El arreglo `vendidas` empieza en 0 y acumula las unidades vendidas de cada una.

```
const int N = 6;

int codigos[N] = {101, 102, 103, 104, 105, 106};
double precios[N] = {1.50, 1.20, 1.00, 3.50, 1.30, 4.00};
int existencias[N] = {20, 15, 4, 8, 0, 10};
int vendidas[N] = {0, 0, 0, 0, 0, 0};
```

Por ejemplo, las Velas están en la posición 2: su código es `codigos[2]` (103), su precio es `precios[2]` (1.00 doblones) y sus existencias son `existencias[2]` (4).

En esta parte todavía no se guardan los nombres: los mensajes identifican cada mercancía por su código. Los nombres se agregan en la Parte 2.

> Cada sección incluye un ejemplo corto de ejecución. El formato puede variar, pero deben aparecer los mismos datos y mensajes.

4.2 Menú principal

El programa muestra un menú que se repite hasta que la tripulación leva anclas:

```
===== BODEGA DEL TIBURON DORADO =====
1. Ver bodega
2. Vender en puerto
3. Reabastecer
4. Buscar por nombre
5. Registrar botin
6. Bitacora
7. Levar anclas
Opcion:
```

- Las opciones 4 y 5 se hacen en la Parte 2. Mientras tanto, pueden mostrar un mensaje como `Disponible en la Parte 2`.
- Una opción que no esté entre 1 y 7 muestra `Opcion invalida` y se vuelve a pedir.

4.3 Ver bodega

Muestra una línea por mercancía con su código, precio y existencias. Todos los montos se muestran con dos decimales. Para lograrlo, agregar `#include <iomanip>` al inicio del archivo y escribir esta línea al comienzo de `main`:

```
std::cout << std::fixed << std::setprecision(2);
```

```
Opcion: 1
Codigo  Precio  Existencias
101     1.50    20
102     1.20    15
103     1.00    4
104     3.50    8
105     1.30    0
106     4.00    10
```

4.4 Vender en puerto

Se pide el código de la mercancía y se busca en la bodega. Si existe, se pide la cantidad que se quiere vender. Se revisan estos casos en orden:

Caso	Qué hace el programa
El código no existe.	Muestra `Codigo invalido` y vuelve al menú, sin pedir la cantidad.
La cantidad es 0 o menos.	Muestra `Cantidad invalida` y vuelve al menú.
La cantidad es mayor que las existencias.	Muestra `Existencias insuficientes` y vuelve al menú.
En otro caso.	Registra la venta.

Al registrar una venta:

- Las existencias de la mercancía disminuyen y sus unidades vendidas aumentan.
- Se muestra el total de la venta (precio × cantidad).
- El total se suma a los doblones ganados y se cuenta una venta más.

```
Opcion: 2
Codigo: 101
Cantidad: 5
101 x5 = 7.50 doblones

Opcion: 2
Codigo: 105
Cantidad: 1
Existencias insuficientes

Opcion: 2
Codigo: 999
Codigo invalido
```

4.5 Reabastecer

Se pide el código de la mercancía y, si existe, la cantidad de unidades que llegan a la bodega:

- Si el código no existe, se muestra `Codigo invalido` y se vuelve al menú, sin pedir la cantidad.
- La cantidad debe estar entre 1 y 100. Si no lo está, se muestra `Cantidad invalida` y se vuelve al menú.
- Si todo es válido, se suman las unidades a las existencias y se muestran las existencias nuevas.

```
Opcion: 3
Codigo: 105
Cantidad: 12
Existencias de 105: 12

Opcion: 3
Codigo: 103
Cantidad: 0
Cantidad invalida
```

4.6 Bitácora y fin del viaje

La opción 6 muestra la bitácora y vuelve al menú. La opción 7 muestra la misma bitácora y termina el programa. La bitácora incluye:

- La cantidad de ventas registradas.
- Los doblones ganados.
- El **valor de la bodega**: la suma de precio × existencias de toda la mercancía.
- La **mercancía más vendida**, la de más unidades vendidas. Si hay empate, se muestra la que aparece primero en la bodega. Si todavía no hay ventas, se muestra `No hubo ventas` en su lugar.
- La lista de mercancía que **escasea**: la que tiene menos de 5 unidades. Si una mercancía tiene 0, se muestra `Agotado` en lugar de la cantidad.

```
Opcion: 7
----- FIN DEL VIAJE -----
Ventas:             3
Doblones ganados:   14.10
Valor de la bodega: 121.50
Mas vendido:        101
Escasea:
  103: 4
```

Si no hubo ventas y no se ha reabastecido nada:

```
Opcion: 6
----- BITACORA -----
Ventas:             0
Doblones ganados:   0.00
Valor de la bodega: 120.00
No hubo ventas
Escasea:
  103: 4
  105: Agotado
```

4.7 Funciones obligatorias de la Parte 1

Función	Qué hace
`int leerEnteroEnRango(int min, int max)`	La misma del Proyecto 2. Debe usarse para la opción del menú.
`int buscarCodigo(int codigos[], int n, int codigo)`	Devuelve la posición del código, o `-1` si no está.
`void mostrarBodega(...)`	Muestra la bodega. Recibe los arreglos que necesite y `n`.
`double valorBodega(...)`	Devuelve el valor de la bodega.
`int masVendido(int vendidas[], int n)`	Devuelve la posición de la mercancía más vendida.

Se pueden agregar otras funciones si ayudan a organizar el programa.

5 Parte 2: La bodega con `struct`

5.1 De arreglos paralelos a un arreglo de `struct`

Los cuatro arreglos de la Parte 1 se reemplazan por un solo arreglo de `struct`:

```
struct Mercancia {
    int codigo;
    std::string nombre;
    double precio;
    int existencias;
    int vendidas;
};
```

- La bodega tiene capacidad para 20 mercancías, guardada en una constante `MAX`. Una variable `n` indica cuántas hay realmente; al zarpar, `n` vale 6.
- Desde esta parte, la bodega, las ventas y la bitácora muestran el nombre de la mercancía.
- Los nombres son de una sola palabra, sin acentos, y se leen con `std::cin >>`.

```
Opcion: 1
Codigo  Nombre      Precio  Existencias
101     Galletas    1.50    20
102     Cuerdas     1.20    15
...

Opcion: 2
Codigo: 101
Cantidad: 5
Galletas x5 = 7.50 doblones

Opcion: 7
----- FIN DEL VIAJE -----
...
Mas vendido:        Galletas
Escasea:
  103 Velas: 4
```

5.2 Buscar por nombre

Se pide un nombre. Si existe una mercancía con ese nombre exacto, se muestran su código, precio y existencias. Si no, se muestra `Mercancia no encontrada`. Las mayúsculas importan: `Brujulas` existe, pero `brujulas` no.

```
Opcion: 4
Nombre: Brujulas
Codigo 106, precio 4.00, existencias 10

Opcion: 4
Nombre: brujulas
Mercancia no encontrada
```

5.3 Registrar botín

Cuando la tripulación captura un botín, se registra como una mercancía nueva al final del arreglo:

- Si la bodega ya tiene `MAX` mercancías, se muestra `Bodega llena` y se vuelve al menú.
- Se pide el código. Si ya existe, se muestra `Codigo repetido` y se vuelve al menú.
- Se piden el nombre, el precio y las existencias iniciales. Las unidades vendidas empiezan en 0.
- Si el precio es 0 o menos, se muestra `Precio invalido` y se vuelve a pedir **solo el precio**.
- Si las existencias son negativas, se muestra `Cantidad invalida` y se vuelven a pedir **solo las existencias**.
- Al terminar, se muestra `Botin registrado` y `n` aumenta en 1.

La mercancía nueva debe aparecer en la bodega y poder venderse, reabastecerse y buscarse como las demás.

```
Opcion: 5
Codigo: 107
Nombre: Loros
Precio: -1
Precio invalido
Precio: 2.5
Existencias: 6
Botin registrado

Opcion: 5
Codigo: 101
Codigo repetido
```

5.4 Funciones obligatorias de la Parte 2

Las funciones de la Parte 1 cambian para recibir el arreglo de `struct`, y se agrega la búsqueda por nombre:

Función	Qué hace
`int leerEnteroEnRango(int min, int max)`	Igual que en la Parte 1.
`int buscarCodigo(Mercancia bodega[], int n, int codigo)`	Devuelve la posición del código, o `-1`.
`int buscarNombre(Mercancia bodega[], int n, std::string nombre)`	Devuelve la posición del nombre, o `-1`.
`void mostrarBodega(Mercancia bodega[], int n)`	Muestra la bodega con los nombres.
`double valorBodega(Mercancia bodega[], int n)`	Devuelve el valor de la bodega.
`int masVendido(Mercancia bodega[], int n)`	Devuelve la posición de la mercancía más vendida.

6 Casos de prueba

El programa debe probarse con todos los casos siguientes antes de entregarlo. Superar estos casos no garantiza la nota completa de implementación, ya que el programa también se evaluará con otros casos.

Junto con este enunciado se entrega el archivo `pruebas.cpp`, que ejecuta automáticamente todos los casos de esta sección. Para usarlo:

1. Colocar `pruebas.cpp` en la misma carpeta que `proyecto3.cpp`.
2. Abrir una terminal en esa carpeta y ejecutar `init.bat`.
3. Compilar y ejecutar el probador:

```
g++ pruebas.cpp -o pruebas.exe
.\pruebas.exe
```

El probador compila `proyecto3.cpp`, lo ejecuta con cada caso de prueba e indica si el caso `PASA` o `FALLA`. Para cada caso que falla, muestra los datos de entrada, qué no se encontró y la salida completa del programa. Como los casos usan los nombres de la mercancía, el probador funciona con el programa completo (Parte 2).

Para que el probador reconozca correctamente la salida del programa, se deben tener en cuenta las siguientes condiciones:

- Todos los montos deben mostrarse con dos decimales. El probador solo reconoce números con decimales, como `7.50` o `0.00`.

- En la bitácora, los montos deben mostrarse en el orden del ejemplo: doblones ganados y luego valor de la bodega.
- Los mensajes deben escribirse igual que en el enunciado, sin acentos (por ejemplo, `Codigo invalido`). No se distingue entre mayúsculas y minúsculas.

Todos los casos terminan con la opción 7, levar anclas.

6.1 Ventas y reabastecimientos

Cada venta o reabastecimiento se indica como `código×cantidad`.

#	Acciones	Resultado esperado
1	Venta `101×5`.	`Galletas x5 = 7.50 doblones`. Al final: ganados 7.50, valor de la bodega 112.50, más vendido Galletas.
2	Venta `106×10`.	`Brujulas x10 = 40.00 doblones`. Al final: ganados 40.00, valor 80.00. Escasean Velas (4), Mapas y Brujulas (`Agotado`).
3	Venta `105×1`.	`Existencias insuficientes`. Al final: `No hubo ventas`, valor 120.00.
4	Venta con código `999`.	`Codigo invalido`, sin pedir la cantidad.
5	Venta `101×0`.	`Cantidad invalida`.
6	Reabastecer `105×12`, `103×0`, `103×101` y código `999`.	Existencias de Mapas: 12; dos veces `Cantidad invalida`; `Codigo invalido`. Al final: valor 135.60, solo Velas escasean.

6.2 Bitácoras

#	Secuencia	Resultado esperado al levar anclas
7	Ventas `101×5`, `102×3`, `101×2` y `103×5`; reabastecer `105×12`.	La venta de `103×5` muestra `Existencias insuficientes`. 3 ventas, ganados 14.10, valor 121.50, más vendido Galletas. Escasea: solo `103 Velas: 4`.
8	Levar anclas sin hacer nada.	0 ventas, ganados 0.00, valor 120.00, `No hubo ventas`. Escasean Velas (4) y Mapas (`Agotado`).
9	Ventas `102×2` y `104×2`.	2 ventas, ganados 9.40, valor 110.60. Más vendido: Cuerdas (empate en 2 unidades; gana la que aparece primero).

6.3 Parte 2

#	Secuencia	Resultado esperado
10	Buscar `Brujulas`; buscar `brujulas`.	Código 106, precio 4.00, existencias 10; luego `Mercancia no encontrada`.
11	Registrar botín con código `107`, nombre `Loros`, precio `-1` y luego `2.5`, existencias `-3` y luego `6`; venta `107×2`.	`Precio invalido`, `Cantidad invalida` y `Botin registrado`; `Loros x2 = 5.00 doblones`. Al final: ganados 5.00, valor 130.00, más vendido Loros, y `107 Loros: 4` escasea.

#	Secuencia	Resultado esperado
12	Registrar botín con código `101`.	`Codigo repetido`, sin pedir el nombre.

7 Restricciones

- Usar únicamente los contenidos de los laboratorios 1 a 7: los del Proyecto 2, arreglos, `struct` y `std::string` leído con `std::cin >>`.
- No usar `getline`, `vector`, punteros ni otras bibliotecas además de `<iostream>`, `<iomanip>` y `<string>`.
- Todo el programa debe estar en un único archivo llamado `proyecto3.cpp`.

8 Entrega

1. Entregar el archivo `proyecto3.cpp` en la fecha y por el medio indicados.
2. Si el estudiante no realiza la presentación, la explicación tiene un valor de 0 y, por lo tanto, la nota final es 0, salvo uso del comodín o retraso justificado según lo establecido en el sílabo.